Image Embeddings: Methods and Considerations

1 Overview

Image embeddings are fixed-dimensional, dense vector representations that capture the semantic and visual content of images. They are used in a variety of tasks such as classification, retrieval, clustering, and more. In this note, we discuss several common approaches to obtaining image embeddings, along with additional practical nuances.

2 Methods for Obtaining Image Embeddings

2.1 Using Pre-trained Convolutional Neural Networks (CNNs)

Description: CNNs trained on large-scale datasets (e.g., ImageNet) learn hierarchical features. Examples include AlexNet, VGG, ResNet, Inception, and EfficientNet.

How It Works:

- An image is fed through the CNN.
- The activations from a later layer (e.g., the last convolutional block or penultimate fully connected layer) are extracted.
- These activations serve as the image embedding.

Advantages:

- Leverages large-scale pre-training.
- Provides robust, general-purpose features.

2.2 Fine-Tuning Pre-trained Networks

Description: Start with a CNN pre-trained on a large dataset and fine-tune it on your specific dataset.

How It Works:

- Replace the final classification layer with one suited to your task.

- Train the network on your data, allowing weights (and hence embeddings) to adapt to domain-specific features.

Advantages:

- Adapts general-purpose embeddings to the nuances of your specific domain.

2.3 Autoencoders and Variational Autoencoders (VAEs)

Description: Autoencoders are unsupervised models that learn to compress and reconstruct images.

How It Works:

- The encoder maps the input image to a lower-dimensional latent vector.
- The decoder reconstructs the image from this latent code.
- The latent vector is used as the image embedding.

VAEs: These add a probabilistic framework, learning a distribution over the latent space. The mean of the latent distribution can serve as the embedding.

Advantages:

- Unsupervised approach that does not require labels.
- The latent space can capture complex structure in the data.

2.4 Self-Supervised and Contrastive Learning Methods

Description: These methods learn embeddings without manual labels by training the network to distinguish between similar and dissimilar images.

How It Works:

- Create augmented versions of an image (e.g., crops, rotations, color distortions).
- Use a contrastive loss (e.g., InfoNCE) to pull together embeddings of different augmentations of the same image while pushing apart embeddings of different images.

Examples: SimCLR, MoCo, BYOL, SwAV, and DINO.

Advantages:

- Do not require labeled data.
- Often achieve performance competitive with supervised methods.

2.5 Vision Transformers (ViT)

Description: ViTs apply Transformer architectures to images by dividing them into patches.

How It Works:

- The image is split into fixed-size patches (e.g., 16×16).
- Each patch is flattened and projected via a linear layer into an embedding space.
- Positional embeddings are added, and the sequence of patch embeddings is processed with self-attention layers.
- The output corresponding to a special classification token (or an aggregation of patch embeddings) serves as the image embedding.

Advantages:

- Captures global relationships via self-attention.
- Scales well with large datasets.

3 Additional Considerations and Nuances

3.1 Hybrid Approaches

In practice, combining methods can yield better embeddings:

- Use CNN-based embeddings together with self-supervised features.
- Combine patch embeddings from ViT with features from a CNN to capture both local texture and global context.

3.2 Dimensionality Reduction and Visualization

After obtaining embeddings, techniques like PCA, t-SNE, or UMAP are often applied:

- **Purpose:** To visualize and understand the structure of the embedding space.
- **Outcome:** Visualizations that show clusters of similar images, aiding in qualitative evaluation.

3.3 Task-Specific Fine-Tuning

Even if pre-trained embeddings are used:

- Fine-tuning the network on task-specific data can significantly improve performance.
- This adapts the embedding space to better capture domain-specific features.

3.4 Domain Adaptation

When there is a domain shift (e.g., using embeddings from ImageNet on medical images):

- Domain adaptation techniques may be necessary to align the source and target embedding distributions.

3.5 Recent Advances in Self-Supervised Learning

Modern self-supervised methods (e.g., BYOL, DINO, SwAV, SimSiam) have pushed the state-of-the-art in learning image embeddings without labels:

- They often produce embeddings that generalize very well across tasks.
- These methods leverage large batch sizes or momentum encoders to learn robust features.

3.6 Evaluation Metrics

The quality of image embeddings can be evaluated in multiple ways:

- **Classification Accuracy:** When embeddings are used in downstream tasks.
- **Retrieval Metrics:** Such as precision, recall, and mean average precision (mAP) when used for image search.
- **Clustering Quality:** Evaluating how well the embeddings group similar images together.

4 Summary

In summary, image embeddings can be obtained via multiple methods:

- **Pre-trained CNNs:** Extract features from a CNN trained on large datasets.
- **Fine-Tuning:** Adapt pre-trained CNNs to the target domain.
- **Autoencoders/VAEs:** Learn a low-dimensional latent space in an unsupervised fashion.
- **Self-Supervised/Contrastive Learning:** Learn embeddings without labels by enforcing similarity between augmented views of the same image.
- **Vision Transformers:** Use transformer architectures applied to image patches.

Additional considerations such as hybrid approaches, visualization via dimensionality reduction, task-specific fine-tuning, domain adaptation, recent self-supervised advances, and evaluation metrics provide further depth in understanding and applying image embeddings effectively.